

CS639 — AI-Assisted Software Development

Study Guide · Lectures 1–4 · condensed for print review

Lecture 1 — Course Introduction & Claude Code

Course & Grading

3 units: U1 AI-Assisted Dev (wk1–5) · U2 Smart Applications/APIs-RAG-agents (wk6–10) · U3 Security/Ethics/Harness Eng (wk11–15). Tue/Thu 75-min, flipped classroom. 5 projects (brownfield P1–P4 + greenfield Final), all via GitLab+CI/CD, auto-graded.

Grade	Wt
Projects	54%
Exams (3×10%)	30%
Participation	8%
Quizzes	7%
Surveys	1%

Drops lowest 5 quiz + 5 activity scores.

Project	Due
P1 Side-Scroller (8%)	Sep 15
P2 Full-Stack Quiz (11%)	Oct 6
P3 RAG Q&A App (11%)	Oct 27
P4 AI Review Agent (11%)	Nov 17
Final Project (13%)	Dec 8

Exams: Midterm1 Oct14 eve (U1) · Midterm2 Nov18 eve (U2) · Final Dec11 5:05–7:05pm (U3). Written, closed-book, **closed-AI**, one 8.5"x11" note sheet.

AI Policy: encouraged — write/debug/refactor/test/explain w/ Claude, iterate on output; other AI tools OK but must be shared manually (Claude Code auto-logs). Not allowed — submitting work you can't explain, letting AI decide unsupervised, AI on quizzes/exams (misconduct).

Tech Stack

Frontend TS+React · Backend Python+FastAPI · DB MySQL+Redis · Docker+Compose · Git/GitLab · GitLab CI/CD · Claude Code (CLI/VS Code/App).

Claude vs. Claude Code

Claude = vision-language model (brain), no file access alone. **Claude Code** = harness (eyes/hands) wrapping it in an agentic loop: explores, edits, runs, self-checks.

Weakness	You do
Ambiguity	State goals/constraints
Correctness	Run/test/read diff
Design drift	Own architecture
Self-awareness	Diagnose/redirect
Higher-order	Bring judgment

3 interfaces: CLI (simplicity) · VS Code (codebase feedback) · App (worktrees/multi-agent/scheduled) — same harness.

@filename for context, /help for commands. **Always read the diff before accepting.** Verify extra when: unfamiliar API, change >few lines, "best way" unexplained, tests pass but you don't know why.

Git Essentials

```
git clone <url>
git checkout -b feature/x
git add <file> && git commit -m "Fix y"
git push -u origin feature/x
```

Messages describe the change, not your actions. **Local merge** (unprotected main): checkout main → merge feature → push. **MR** (protected main): push → open MR → teammate approves (not you) → CI green → merge. **Fork sync:** git fetch upstream && git merge upstream/main.

Gotcha: Claude Code loads .claude/settings.json/rules//skills/ only from the exact launch folder — open each lecture/project folder directly, never the repo root.

Lecture 2 — Claude Code in Practice

Java → TS & p5.js

Same OOP concepts (classes/interfaces/access modifiers), syntax differs. p5.js: setup() runs once (createCanvas), draw() runs ~60x/sec (input→update→render) — same shape as Java's init()→loop.

Commands

/help commands · @file context · /compact shrink session · /clear fresh session · /context window usage · /usage tokens/plan.

Reviewing Edits & Bash

Claude proposes a diff first — check: expected files only? matches ask? smaller way? **Never blind-Accept.** Claude can run terminal commands with approval (most modes) — good for driving the local loop ("run tests, tell me what fails").

Unfamiliar codebase: ask before touching ("overview this") → trace behavior ("what happens in one frame?") → ask about anything strange → then propose change & iterate.

AAA Testing

```
function formatScore(s:number){ return
s.toString().padStart(4,"0") }
// formatScore(7) -> "0007"
describe("formatScore", () => {
  it("pads to 4 digits", () =>
expect(formatScore(7)).toBe("0007"));
});
```

Local dev loop: npm run test (vitest, must pass) → format (prettier auto-fix) → lint (eslint, fix errors) → commit → push. Same checks CI runs — catch failures in seconds, not a push/wait/fix round trip.

Lecture 3 — Brownfield Development

Two Memory Systems

	CLAUDE.md	Auto memory
Writer	You	Claude
Has	Instructions/rules	Learnings/patterns

Both via /memory. Scopes broad→specific: Managed policy (IT) → User ~/.claude/CLAUDE.md → Project ./CLAUDE.md (team, git) → Local ./CLAUDE.local.md (gitignored). Nested CLAUDE.md loads on-demand per subdir. Auto memory: ~/.claude/projects/<proj>/memory/. Auto-loads every session: CLAUDE.md, CLAUDE.local.md, auto memory, .claude/rules/*.md, settings.json. **Doesn't:** root RULES.md/SPEC.md — needs See @SPEC.md for... in CLAUDE.md to count. /context: prompt+memory+skills <3% combined; Messages = only non-auto-loaded, largest, category.

Rules & Skills

```
---
paths: ["src/**/*.ts"]
---
# create tests for modified functions
```

No paths: → loads every session. Scoped → only when matching file touched. **Skill** = .claude/skills/<name>/SKILL.md, auto-invoked or via /name. /init drafts a starter CLAUDE.md; /help beats stale docs.

settings.json

Scopes: User ~/.claude/settings.json (all projects) → Project .claude/settings.json (team, git) → Local .claude/settings.local.json (gitignored) — most specific wins. **Permissions:** allow(silent)/ask(prompt)/deny(refuse), e.g. "Bash (npm run test *)". **env:** auto-injects vars every session.

Hooks = enforcement (vs. CLAUDE.md/rules = context, no guarantee). A hook runs on a tool event regardless, can block: e.g. PostToolUse on Edit|Write → run npm run format, no opt-out. If it must happen every time → hook, not an instruction.

Merge Conflicts

```
<<<<<<< HEAD
export const X = 1200;
=====
export const X = 600;
>>>>>> branch-name
```

Happens when both branches touch the same lines. Above ====yours, below=incoming. Pick/combine/rewrite, remove markers.

Never rebase a graded repo: git rebase/git pull —rebase delete merge commits graders check for. Safe: plain git merge/git pull. GitLab "Update fork" already merges safely.

Lecture 4 — Elephant-Goldfish Model (EGM)

Why a Process

No-process assistant = **elephant with amnesia:** huge capability, no cross-session memory, no sense of "done." Failure modes: **over-ambition** (runs out of context) / **premature victory** (declares done early). Fix: Elephant forgets → build durable external memory. Goldfish (fresh context remembers nothing) → use that blankness as a test.

Four Phases

(0) **Context hierarchy** (prereq, build once/refresh) → (1) **Growing the Elephant** (load context, discuss design, no code) → (2) **Teaching the Elephant** (write SPEC.md) → (3) **Goldfish Protocol** (test doc, fresh/context-free session) → (4) **Implementation** (build, then mean-review).

Permission Modes (Shift+Tab)

Mode	Runs unasked
Auto (default)	Everything — classifier reviews
Manual	Nothing — OK each time
Accept Edits	Edits auto-apply, check diff after
Plan	Reads only, no edits till approved

Phases 1–3 need explicit **Plan Mode** (/plan) — only mode where Claude is physically blocked from editing. Ctrl+G opens plan in editor → draft SPEC.md.

P1 Growing the Elephant

Load context → No Code Rule → first draft in prose. **Sycophant Challenge** (push back on Claude's own proposal): "What could go wrong?" / "What edge cases are missing?" / "Simpler way?"

Context Hierarchy

Recursive summarization, **leaves→root**, as nested CLAUDE.md (not README, so it auto-loads): summarize deepest dirs first → verify → one level up, summarize the summaries (not raw source) → repeat to root. Sync via a .claude/rules/ entry. Same trick as reading docs instead of a whole library's source.

P2 Teaching the Elephant

SPEC.md sections: **Problem** (1-2 sentences) · **Technical Plan** · **Alternatives** (+why they lost) · **Detailed Implementation**. One file per feature in docs/plans/. Test: could someone with zero context implement from the doc alone?

P3 Goldfish Protocol

Feed spec+hierarchy to a brand-new session (/clear), never the discussion that made them. Checks: comprehension (explains it back?) · critic (what holes?) · readiness (knows exactly what to build?). Gaps → revise spec before implementing. Stay in Plan Mode.

P4 Implementation & Beyond

Only now write code, against the goldfish-tested spec; verify against tests as you go. **Mean review:** have Claude tear apart its own "done" work — critical posture catches what generation doesn't. From P2 on: context hierarchy is self-written bottom-up; same design+SPEC+fresh-test; implementation adds full CI/CD.